GITHUB ACTIONS

Master Notes - Easy Language + Real-World Examples + Dry Runs

Learning style	Tree hierarchy: Part A -> A.1 -> A.1.a
Coverage	Complete source material + clearly marked essential additions
Code policy	Broken examples corrected; deprecated syntax replaced where current docs require it
Reference point	GitHub Actions practices checked against official GitHub documentation in October 2026

How to use this PDF: Read Part A once for the mental model, then work through the remaining parts in order. For revision, focus on the tree, the key idea, the real-world example, and the dry run after each important code block.

Important source handling: this PDF is based on the supplied GitHub Actions course material. Where the source was incomplete, typo-ridden, outdated, or unsafe, the note explicitly labels the improvement as a correction or an added topic.

Master Learning Tree

The complete path is arranged from fundamentals to production-oriented security. The original course sections are preserved, while the hierarchy makes the dependency between concepts clearer.

```
PART A - FOUNDATION
|
+-- A.1 GitHub
|   +-- repositories
|   +-- collaboration
|   +-- automation
|
+-- A.2 Git
|   +-- commits
|   +-- branches
|   +-- checkout
|
+-- A.3 GitHub Actions
|   +-- workflow automation
|   +-- CI/CD
|
PART B - ACTIONS FUNDAMENTALS
|
+-- B.1 Workflow
+-- B.2 Job
+-- B.3 Step
+-- B.4 Runner
+-- B.5 Actions vs run commands
+-- B.6 Basic workflow
+-- B.7 Multiple jobs and needs
+-- B.8 Expressions and contexts
|
PART C - WORKFLOWS AND EVENTS
|
+-- C.1 Events
+-- C.2 Activity types
+-- C.3 Branch filters
+-- C.4 Path filters
+-- C.5 Fork pull requests
+-- C.6 Cancellation and skipping
|
PART D - ARTIFACTS, OUTPUTS, CACHING
|
+-- D.1 Artifacts
+-- D.2 Job outputs
+-- D.3 Step outputs and GITHUB_OUTPUT
+-- D.4 Dependency caching
|
PART E - ENVIRONMENT DATA
|
+-- E.1 env variables
+-- E.2 vars / configuration variables
+-- E.3 Secrets
+-- E.4 Repository environments
|
PART F - EXECUTION CONTROL
|
+-- F.1 if conditions
+-- F.2 failure / success / always / cancelled
+-- F.3 continue-on-error
+-- F.4 outcome vs conclusion
|
PART G - MATRIX STRATEGIES
|
+-- G.1 Matrix expansion
+-- G.2 include
```

```
+-- G.3 exclude
+-- G.4 Failure behavior
|
PART H - REUSABLE WORKFLOWS
|
+-- H.1 workflow_call
+-- H.2 inputs
+-- H.3 secrets
+-- H.4 outputs
+-- H.5 caller vs called workflow
|
PART I - DOCKER CONTAINERS
|
+-- I.1 Job containers
+-- I.2 Container image / Dockerfile
+-- I.3 Service containers
+-- I.4 Networking
|
PART J - CUSTOM ACTIONS
|
+-- J.1 Composite actions
+-- J.2 JavaScript actions
+-- J.3 Docker actions
+-- J.4 Inputs and outputs
+-- J.5 AWS S3 deployment example
|
PART K - SECURITY AND PERMISSIONS
|
+-- K.1 Script injection
+-- K.2 Third-party actions
+-- K.3 Least privilege
+-- K.4 GITHUB_TOKEN and permissions
+-- K.5 OIDC for cloud access
+-- K.6 Security checklist
|
PART L - ESSENTIAL ADDITIONS
|
+-- L.1 Modern action versions
+-- L.2 Concurrency
+-- L.3 Artifacts vs cache vs outputs
+-- L.4 Troubleshooting
+-- L.5 End-to-end CI/CD blueprint
```

Source vs added material

- Source-derived: all core topics visible in the supplied material, including the examples around workflows, jobs, steps, events, artifacts, outputs, caching, env/secrets, execution control, matrices, reusable workflows, containers, custom actions, and security.
- Corrected: spelling/syntax mistakes and obsolete workflow techniques that would otherwise confuse a learner.
- Added for completeness: concepts that are needed to use the source material correctly in modern GitHub Actions, clearly labeled so they are not mistaken for the original notes.

PART A | Foundation: GitHub, Git, GitHub Actions and CI/CD

Build the mental model before writing YAML.

A.1 What is GitHub?

GitHub is a cloud platform for storing and working with Git repositories. It is more than file storage: it adds collaboration, code-review, project-management, and automation capabilities on top of Git.

Capability	What it means	Real-world example
Repository storage	Store Git repositories online and push/pull changes.	Your Java Spring Boot project is stored privately on GitHub.
Collaboration	Issues, pull requests, projects, discussions and review workflows.	A teammate opens a pull request for a bug fix and you review it before merge.
Automation	GitHub Actions and apps can react to repository events.	A push starts tests automatically.

A.1.a GitHub repository

A repository is the project home on GitHub. It can contain source code, configuration, documentation, issues, pull requests, workflows, releases, and other project information.

A.1.b Easy way to remember

Think of it like this: Git is the version-control engine. GitHub is a platform built around Git that gives the team a shared online home plus collaboration and automation tools.

A.2 What is Git?

Git is a free distributed version-control system used to track source-code changes.

- Commit = save a snapshot of the project.
- Branch = work on an alternative line of development.
- Checkout/switch = move to another branch or commit.
- History = return to an older snapshot when needed.

A.2.a Real-world example

You are developing a shopping application. The main branch contains stable code. You create feature/cart, make commits, test the feature, and merge it only after review. If the new version breaks, Git lets you inspect or restore an earlier commit.

A.2.b Git vs GitHub

Git	GitHub
Tool installed on your machine	Cloud service/platform
Tracks versions locally	Hosts and shares Git repositories
Commit, branch, merge, rebase	Pull requests, issues, Actions, collaboration

A.3 What is GitHub Actions?

GitHub Actions is GitHub's workflow automation service. A workflow can react to repository events or other triggers and execute jobs that test, build, package, deploy, report, or automate repository tasks.

A.3.a Typical CI/CD flow

```
Developer pushes code
        |
        v
   Workflow starts
        |
        +----> Checkout code
        |
        +----> Install dependencies
        |
        +----> Lint
        |
        +----> Test
        |
        +----> Build
        |
        +----> Deploy
```

A.3.b What can be automated?

- Code testing, building, packaging and deployment.
- Code review helpers, issue management, labels, release tasks and repository automation.
- Scheduled maintenance or manually triggered workflows.

A.4 CI/CD

A.4.a Continuous Integration (CI)

CI means code changes are integrated frequently and verified automatically by building and testing them. The goal is to find integration problems early.

A.4.b Continuous Delivery (CD)

In the supplied notes, Continuous Delivery is described as automatically publishing a new application or package version after integration. In practice, CD commonly means the software is kept in a releasable state and can be delivered automatically, often through a controlled deployment pipeline.

A.4.c Real-world example

Imagine 10 developers working on an e-commerce application. Every pull request runs lint + unit tests. After merge to main, a build is created. After approval, the build is deployed to a staging environment. Production deployment can then be automatic or protected by approvals.

> Memory trick: CI = integrate + verify. CD = keep the verified software ready to deliver/deploy.

A.5 Core terminology

Term	Simple meaning
Workflow	The complete automation definition in a YAML file.
Job	One unit of work running on a runner.
Step	One action or shell command inside a job.
Runner	The machine/environment that executes a job.
Action	Reusable automation packaged as a component.
Event	What causes a workflow to start.
Artifact	Files/folders saved by a workflow run.
Output	Small value passed from a step/job to later work.
Cache	Reusable dependency data kept to speed up future runs.

A.5.a Essential source correction

Correction: The source contains several examples written with action/checkout instead of actions/checkout, run-on instead of runs-on, need instead of needs, and ubutu-latest instead of ubuntu-latest. These are YAML/workflow errors and are corrected throughout this PDF.

PART B | GitHub Actions Fundamentals

Understand the three-layer structure: workflow -> job -> step.

B.1 Workflow

A workflow is a YAML file stored in `.github/workflows/`. It defines when automation starts and what jobs should run.

B.1.a Basic workflow skeleton

```
name: Hello World

on:
  workflow_dispatch:

jobs:
  first-job:
    runs-on: ubuntu-latest
    steps:
      - name: Print greeting
        run: echo "Hello world"

      - name: Print goodbye
        run: echo "Done - bye!"
```

Dry run: 1) You manually start the workflow because of `workflow_dispatch`. 2) GitHub creates a runner for `first-job`. 3) The first step prints `Hello world`. 4) The second step prints `Done - bye!`. 5) The job finishes successfully if both commands exit with code 0.

B.2 Jobs

A workflow contains one or more jobs. Each job defines its execution environment with `runs-on` and contains one or more steps.

- Jobs run in parallel by default when no dependency is declared.
- Jobs can be made sequential with `needs`.
- Jobs can have conditions with `if`.

B.2.a Runner isolation

The source correctly emphasizes an important idea: every job gets its own runner. You should not assume that files created by one job automatically exist in another job.

Real-world example: Job `test` creates a report. Job `deploy` needs that report. Because the jobs have separate runners, `deploy` should download the report as an artifact or recreate the file.

B.3 Steps

Steps are executed in order inside a job. A step can run a shell command with `run` or execute a reusable action with `uses`. Steps may also be conditional.

Syntax	Purpose	Example
run	Execute your shell command/script.	`run: npm test`
uses	Call an action.	`uses: actions/checkout@v6`
with	Pass inputs/configuration to an action.	`with: node-version: '20'`
id	Give a step a name for outputs/context.	`id: build`

Syntax	Purpose	Example
if	Conditionally execute a step.	`if: ${{ ... }}`

B.4 Actions

An action is a reusable package that performs a task. Actions can be created by GitHub, the community, or you.

B.4.a Action vs command

Action	Shell command
Reusable packaged component	A command/script you define
Often handles complex repeated tasks	Usually simple and direct
Example: checkout repository	Example: `npm test`

B.4.b Real-world example

Instead of manually downloading the repository in every workflow, use `actions/checkout`. Instead of writing custom Node setup logic, use `actions/setup-node`. Use `run` when the exact command belongs to your project.

B.5 Events / workflow triggers

An event is what starts a workflow. The source lists repository events such as `push`, `pull_request`, `create`, `fork`, `issues`, `issue_comment`, `watch`, and `discussion`, plus non-activity style triggers such as `workflow_dispatch`, `repository_dispatch`, `schedule`, and `workflow_call`. GitHub supports many more events.

B.5.a Most important triggers for beginners

Trigger	Use it when...
push	Code is pushed to a branch.
pull_request	A pull request is opened/updated/etc.
workflow_dispatch	You want a manual button to start it.
schedule	You want time-based automation.
workflow_call	Another workflow should call this workflow.

B.6 Basic project setup

B.6.a GitHub UI route

- Create a GitHub repository.
- Open the Actions tab.
- Choose a starter workflow/template or create a YAML workflow.
- Commit the workflow file.

B.6.b Local project route

- Create/push the project to GitHub.
- Create `.github/workflows/<name>.yml`.
- Add the workflow YAML.

- Push the workflow file.

The source mentions generating a personal token to push workflow changes. Modern GitHub workflows normally use the automatically generated `GITHUB_TOKEN` for workflow authentication rather than creating a personal token for ordinary repository API operations. A personal token or GitHub App can still be needed for special cases, such as cross-repository access not available to `GITHUB_TOKEN`.

B.7 Full beginner CI example

```
name: Test Project

on:
  push:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Get code
        uses: actions/checkout@v6

      - name: Install Node.js
        uses: actions/setup-node@v7
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test
```

Dry run: Push -> workflow starts -> `checkout` places the repository on the runner -> `setup-node` selects Node 20 -> `npm ci` installs from the lock file -> `npm test` runs tests. If any step fails, the job fails and later normal steps do not run.

Source correction: The original notes used older v3 action references. Current GitHub documentation examples use newer major versions such as checkout v6 and setup-node v7. Always check the action's current release before copying a version number into production.

B.8 Multiple jobs

```
name: Deploy Project

on:
  push:
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - run: npm ci
      - run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - run: npm ci
      - run: npm run build
      - run: echo "Deploying..."
```

Dry run without needs: `test` and `deploy` can start independently. Dry run with `needs: test`: `deploy` waits for `test`. If `test` fails, `deploy` is skipped unless you deliberately change the condition.

B.9 Expressions and contexts

Expressions let a workflow calculate or look up values dynamically. Contexts are structured objects containing information about the workflow run, repository, event, job, matrix, steps, secrets, variables and more.

```
name: Output information

on:
  workflow_dispatch:

jobs:
  info:
    runs-on: ubuntu-latest
    steps:
      - name: Output GitHub context
        run: echo '${{ toJSON(github) }}'
```

Dry run: `github` is a context object containing metadata about the run, repository and triggering event. `toJSON(github)` converts that object into printable JSON. Use context data carefully - do not print secrets or sensitive data.

B.9.a Core expression examples

Expression	Meaning
`` `${{ github.ref }}` ``	Reference that triggered the run, such as a branch or tag.
`` `${{ github.sha }}` ``	Commit SHA associated with the run.
`` `${{ github.repository }}` ``	Owner/repository name.
`` `${{ steps.build.outputs.name }}` ``	Output named `name` from step with id `build`.
`` `${{ needs.build.outputs.file }}` ``	Job output from a dependent `build` job.
`` `${{ matrix.node-version }}` ``	Current matrix value.

Added for completeness: Context availability depends on where an expression is used. GitHub documents which contexts/functions are valid for specific workflow keys. Do not assume every context is available everywhere.

PART C | Workflows and Events - Deep Dive

Control exactly when a workflow should run.

C.1 Why event control matters

A workflow that runs on every event may waste time or create noisy results. Filters let you say exactly which branches, paths, or activity types should trigger a run.

C.1.a Event -> activity type -> filter

```
Event
  |
  +-- Activity types
  |     +-- opened
  |     +-- edited
  |     +-- closed
  |
  +-- Filters
        +-- branches
        +-- paths
        +-- branches-ignore
        +-- paths-ignore
```

C.2 Activity types

Some events have multiple activity types. Use `types` to narrow down which activity causes the workflow to run. For example, a pull request workflow might run only when a PR is opened, edited, or reopened.

```yaml
on:
  pull_request:
    types: [opened, edited, reopened]
```

Dry run: a PR opened event matches -> workflow runs. A PR `closed` event does not match this `types` list -> workflow does not start from that event.

C.3 Branch filters

```yaml
on:
  push:
    branches:
      - main
      - 'dev-*'
      - 'feat/**'
```

Dry run: push to `main` -> yes. Push to `dev-login` -> yes. Push to `feat/auth/login` -> yes because the pattern covers nested names. Push to `release` -> no.

C.3.a Common patterns

Pattern	Example match
`main`	main
`dev-*`	dev-login, dev-api
`feat/**`	feat/auth, feat/auth/login
`branches-ignore`	Run everywhere except named branches.

C.4 Path filters

```
on:
  push:
    branches:
      - main
    paths:
      - '.github/workflows/**'
```

This workflow runs only when a push targets `main` and the changed files match the path filter. A change only to `src/app.js` does not match this example.

C.4.a paths-ignore

```
on:
  push:
    paths-ignore:
      - '.github/workflows/**'
```

This is the opposite style: the workflow is skipped when the push changes only the ignored paths. The exact filter logic should be read as an inclusion/exclusion rule, not as a generic 'ignore anything related' shortcut.

C.5 Pull requests from forks

The source warns that fork-based pull requests require special handling because anyone may create a fork and attempt to run automation. Modern GitHub applies restrictions around secrets, permissions, and approval for workflows from forks. Exact behavior depends on the event (`pull_request` vs `pull_request_target`) and repository settings.

Security reason: A workflow triggered by untrusted contribution code can become an attack path to repository secrets or write-capable tokens. Treat fork PR code as untrusted input.

C.6 Cancellation and skipping

C.6.a What happens when a step fails?

By default, later ordinary steps in the same job are skipped after a failure. A job normally has a failure result when one of its steps fails.

C.6.b Important correction to the source

Correction: It is not accurate to say that every workflow is automatically cancelled whenever one job fails. Independent jobs can continue. Jobs that depend on a failed job are normally skipped. You can also manually cancel a workflow run and can use conditional functions to decide what should still run.

C.6.c Skipping

Workflows are triggered only for events that match their trigger definition and filters. For push and `pull_request`, branch/path filters determine whether the workflow should run.

C.7 Real-world design

Use event filters to reduce noise and cost. Example: run quick CI for every pull request, run deployment only on pushes to `main`, and run a nightly maintenance workflow with `schedule`. This separation makes the pipeline easier to understand.

PART D | Artifacts, Job Outputs and Dependency Caching

Understand what is shared because jobs do not share a filesystem.

D.1 Artifacts

Artifacts are files and folders produced during a workflow run that you want to keep after a job completes or share with another job. Typical examples: compiled packages, test reports, coverage output, logs, screenshots, binaries, and deployment bundles.

D.1.a Artifact mental model

```
Build job
   |
   +--> dist/
   +--> test-report/
   |
   +--> upload-artifact
          |
          v
      GitHub stores files
          |
          +--> download-artifact in later job
          |
          +--> manual download after the run
```

D.1.b Upload artifact - current form

```
- name: Upload build
  uses: actions/upload-artifact@v4
  with:
    name: dist-files
    path: |
      dist
      package.json
```

Dry run: the action takes the `dist` directory and `package.json`, packages them under the artifact name `dist-files`, and stores them with the workflow run.

D.1.c Download artifact

```
- name: Download build
  uses: actions/download-artifact@v5
  with:
    name: dist-files
```

The later job receives the stored files in its workspace. Because a new runner is used, this is how the built output crosses the job boundary.

Source correction: The supplied material used upload/download artifact v3. Current official examples use upload-artifact v4 and download-artifact v5.

D.2 Full build -> deploy example

```yaml
name: Deploy website

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v7
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run lint
      - run: npm test

  build:
    needs: test
    runs-on: ubuntu-latest
    outputs:
      script-file: ${{ steps.publish.outputs.script-file }}
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v7
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run build

      - name: Publish JS filename
        id: publish
        shell: bash
        run: |
          file=$(find dist/assets -maxdepth 1 -name '*.js' -type f | head -n 1)
          echo "script-file=$file" >> "$GITHUB_OUTPUT"

      - name: Upload build artifact
        uses: actions/upload-artifact@v4
        with:
          name: dist-files
          path: dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Download build artifact
        uses: actions/download-artifact@v5
        with:
          name: dist-files

      - name: Show files
        run: ls -la

      - name: Show script file output
        run: echo "Script file = ${{ needs.build.outputs.script-file }}"

      - name: Deploy
        run: echo "Deploying..."
```

Dry run: `test` must pass -> `build` starts -> build generates `dist` -> `publish` writes one value to `$GITHUB_OUTPUT` -> the job maps that step output to the job output `script-file` -> artifact is uploaded -> `deploy` waits for `build`, downloads `dist-files`, then reads `needs.build.outputs.script-file`.

D.3 Job outputs

Job outputs are small values exposed by one job to downstream jobs. They are not a replacement for files.

Use output when...	Use artifact when...
You need a small value.	You need files/folders.
Example: version number.	Example: JAR, ZIP, dist/ directory.
Example: generated filename.	Example: test report.

D.3.a Step output -> job output

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.info.outputs.version }}
    steps:
      - id: info
        run: echo "version=1.4.0" >> "$GITHUB_OUTPUT"

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploy version ${{ needs.build.outputs.version }}"
```

Dry run: step `info` creates the text value `1.4.0` -> job output `version` points to it -> `deploy` can read `needs.build.outputs.version` after `build` completes.

D.4 How `$GITHUB_OUTPUT` works

A step with an `id` can publish named output values by writing `name=value` into the special `GITHUB_OUTPUT` environment file.

```
- name: Set version
  id: release
  run: echo "version=1.4.0" >> "$GITHUB_OUTPUT"

- name: Read version
  run: echo "${{ steps.release.outputs.version }}"
```

Source correction: `::set-output` is obsolete. Use `$GITHUB_OUTPUT` for step outputs. This applies to custom actions and workflow `run` steps alike.

D.5 Dependency caching

Dependencies usually do not change on every commit, but downloading them repeatedly costs time. A dependency cache stores reusable package-manager data so later runs can restore it quickly.

D.5.a Source mental model

```
Without cache
Get code -> install dependencies -> test
Get code -> install dependencies -> build

With cache
Get code -> restore cache -> install only if needed -> test/build
```

D.5.b Cache with setup-node

```
steps:
  - uses: actions/checkout@v6
  - uses: actions/setup-node@v7
    with:
      node-version: '20'
      cache: 'npm'
  - run: npm ci
  - run: npm test
```

This is a simple modern pattern. `setup-node` can cache the package manager's cache for npm rather than treating `node_modules` as the default cache target.

D.5.c Manual cache

```
- name: Cache npm
  id: cache-npm
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-
```

Dry run: the cache key includes the OS and the lock-file hash. Same dependencies -> same key -> exact hit. Change `package-lock.json` -> new hash -> new cache key. `restore-keys` can fall back to a broader prefix.

D.5.d Why `hashFiles` is useful

The dependency definition is part of the cache key. When `package-lock.json` changes, the hash changes, so the workflow does not incorrectly reuse an old dependency state.

Security note: Do not store access tokens, credentials, or other sensitive files inside dependency caches. GitHub explicitly warns that caches can be readable by contributors in relevant pull request scenarios.

PART E | Environment Variables, Variables, Secrets and Environments

Control dynamic configuration without hard-coding it.

E.1 Environment variables

Environment variables hold dynamic configuration values used by commands or actions. In GitHub Actions, env can be defined at workflow, job, or step scope.

```
name: Deploy website

on:
  push:
    branches: [main, dev]

env:
  MONGODB_DB_NAME: gha-demo

jobs:
  test:
    runs-on: ubuntu-latest
    env:
      MONGODB_CLUSTER_ADDRESS: example.cluster.mongodb.net
      MONGODB_USERNAME: test-user
      PORT: 8080
    steps:
      - uses: actions/checkout@v6
      - name: Show username
        run: echo "MONGODB_USERNAME=$MONGODB_USERNAME"
```

Dry run: `MONGODB_DB_NAME` is available to all jobs. Job-level values are available only in the `test` job. The shell sees the environment variable using shell syntax such as `$MONGODB_USERNAME` on Linux.

E.1.a Scope and precedence

Scope	Available to
Workflow env	All jobs/steps in that workflow unless overridden.
Job env	All steps in one job.
Step env	Only that step.

A more specific definition overrides a broader one for the time it is in scope.

E.2 `env` vs `vars`

The source focuses mainly on env, while modern GitHub also provides configuration variables exposed through the `vars` context. These are useful when values need to be configured centrally at repository, organization, or environment level.

Use	Access pattern	Good example
Environment variable	`${{ env.PORT }}` in expression context; `$PORT` in shell.	PORT=8080 for a test server.
Configuration variable	`${{ vars.API_BASE_URL }}`	Environment-specific non-secret URL.

E.3 Secrets

Secrets are values that should not be exposed, such as database passwords, API keys, signing material, and deployment credentials.

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        env:
          MONGODB_PASSWORD: ${{ secrets.MONGODB_PASSWORD }}
        run: npm test
```

Dry run: GitHub injects the secret into the step environment. The code can use it as `$MONGODB_PASSWORD`. Do not hard-code the password in the YAML file or Dockerfile.

Critical correction: The supplied source contains hard-coded password examples. Those values are unsuitable for a real repository. This PDF replaces them with placeholders and shows the secret-based pattern.

E.3.a Secrets vs env

env variable	Secret
May be non-sensitive	Designed for sensitive data
Can be committed in workflow YAML if safe	Should be stored in GitHub's secret store
Example: PORT=8080	Example: database password

E.3.b Secret masking is not a security strategy by itself

GitHub attempts to redact known secret values from logs, but transformed or derived values can still leak. Treat secrecy as an architectural rule: keep credentials out of logs, files, caches, artifacts, command arguments where possible, and source code.

E.4 Repository environments

A GitHub Environment groups deployment-related configuration and protection rules. A job can target an environment, which allows that environment's secrets and variables to be scoped to jobs using it.

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: echo "Deploying to production"
```

E.4.a Why environments matter

- Separate development/testing/production configuration.
- Store different environment secrets, such as different database passwords.
- Protect production with required reviewers and deployment branch/tag rules.
- Use environment approval before a sensitive deployment proceeds.

E.4.b Example

```
Repository
|
+-- testing environment
|   +-- test DB secret
|   +-- test URL
|
+-- production environment
    +-- prod DB secret
    +-- prod URL
    +-- required reviewer
    +-- allowed deployment branch = main
```

Dry run: a job targeting `production` cannot use protected environment secrets until the configured rules are satisfied. This gives a clean gate between 'code built' and 'production changed'.

Added for completeness: Environment variables and configuration variables are different concepts. Environment-level configuration variables are accessed with the `vars` context, while `env` values are environment variables defined in the workflow.

PART F | Controlling Workflow and Job Execution

Use conditions to turn a linear workflow into a decision-making pipeline.

F.1 The default execution model

A step normally runs only when earlier required work succeeds. A job that needs another job normally waits for that dependency to succeed. Conditions let you override this default behavior for specific cases such as report collection or cleanup.

F.2 `if` conditions

```
- name: Install dependencies
  if: steps.cache.outputs.cache-hit != 'true'
  run: npm ci
```

Dry run: cache hit is `true` -> condition becomes false -> installation step is skipped. Cache miss -> condition becomes true -> dependencies are installed.

F.2.a Step condition

Put `if` on an individual step when only that step should react to a condition.

F.2.b Job condition

Put `if` on the job when the whole job should be conditional.

F.3 Special conditional functions

Function	Meaning	Typical use
failure()	True when a previous required step/job failed.	Upload failure report.
success()	True when the workflow is in a successful state for the evaluated scope.	Run normal post-success logic.
always()	Runs regardless of success/failure/cancellation outcome within the supported context.	Cleanup or final reporting.
cancelled()	True when the workflow/job was cancelled.	Cleanup or cancellation-specific logic.

F.4 Upload a test report after a failing step

```
- name: Run tests
  id: run-tests
  run: npm test

- name: Upload test report
  if: ${{ failure() && steps.run-tests.outcome == 'failure' }}
  uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: test.json
```

Dry run: suppose `npm test` exits 1. The `run-tests` outcome is `failure`. `failure()` is true, so the report-upload step runs even though the prior step failed.

F.4.a `outcome` vs `conclusion`

This distinction becomes important with `continue-on-error`. A failed step can have `outcome: failure` while its final conclusion is treated as successful for job flow purposes. That lets you continue while still knowing that the step itself failed.

F.5 `continue-on-error`

```
- name: Run experimental tests
  continue-on-error: true
  id: run-tests
  run: npm test

- name: Continue working
  run: echo "The job continues"
```

Dry run: `npm test` fails -> the step outcome is failure -> `continue-on-error` prevents the failure from stopping the job -> the next step executes. This is useful for non-critical checks, experiments, or test jobs where you still want later diagnostics.

Caution: Do not use `continue-on-error` to hide real production failures. If deployment silently continues after a broken build, the pipeline becomes dangerous rather than resilient.

F.6 Failure-report job

```
jobs:
  report:
    needs: [lint, build, deploy]
    if: ${{ failure() }}
    runs-on: ubuntu-latest
    steps:
      - name: Report failure
        run: |
          echo "Something went wrong"
          echo '${{ toJSON(github) }}'
```

Dry run: if any of the required upstream jobs fails and the report job is otherwise eligible, the report job can start because `failure()` changes the default success-only behavior.

F.7 Condition design checklist

- Ask: should this run only after success, after failure, always, or only on cancellation?
- Use an explicit `id` when later steps need a step output/outcome.
- Use `needs` when a job needs another job's output or status.
- Prefer small, readable expressions over one huge `if` statement.

PART G | Matrix Strategies

Run the same job across many supported configurations.

G.1 What is a matrix?

A matrix generates multiple copies of the same job using different configuration combinations. It is useful for testing multiple versions of a language/runtime and multiple operating systems.

```
name: Matrix Demo

on:
  push:

jobs:
  build:
    strategy:
      matrix:
        node-version: [18, 20, 22]
        operating-system: [ubuntu-latest, windows-latest]

    runs-on: ${{ matrix.operating-system }}

    steps:
      - uses: actions/checkout@v6

      - name: Install Node.js
        uses: actions/setup-node@v7
        with:
          node-version: ${{ matrix.node-version }}

      - run: npm ci
      - run: npm run build
```

Dry run: 3 Node versions x 2 operating systems = 6 base combinations. GitHub creates separate matrix job runs for each combination.

G.1.a Why it is useful

Without a matrix, you would write six similar jobs by hand. With a matrix, you write one job and let GitHub expand it.

G.2 `include`

```
strategy:
  matrix:
    node-version: [18, 20, 22]
    operating-system: [ubuntu-latest, windows-latest]
    include:
      - node-version: 24
        operating-system: ubuntu-latest
```

Here the matrix includes an extra combination beyond the base Cartesian product. Think of `include` as a way to add or enrich specific combinations.

G.3 `exclude`

```
strategy:
  matrix:
    node-version: [18, 20, 22]
    operating-system: [ubuntu-latest, windows-latest]
    exclude:
      - node-version: 18
        operating-system: windows-latest
```

Dry run: the base matrix produces 6 combinations; `exclude` removes one -> 5 combinations remain.

G.4 `continue-on-error` and matrices

The supplied notes mention using `continue-on-error` to stop one experimental matrix combination from taking down all other combinations. This is useful when a particular version is allowed to fail or is intentionally experimental.

```
jobs:
  build:
    continue-on-error: true
    strategy:
      matrix:
        node-version: [18, 20, 22]
        operating-system: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.operating-system }}
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v7
        with:
          node-version: ${{ matrix.node-version }}
      - run: npm ci
      - run: npm run build
```

Use this deliberately. For a production support matrix, it is usually better to keep the matrix strict and identify exactly which configurations are allowed to fail rather than ignore all failures.

G.5 Matrix dry-run table

#	Node	OS	Result
1	18	Ubuntu	Base
2	18	Windows	Base
3	20	Ubuntu	Base
4	20	Windows	Base
5	22	Ubuntu	Base
6	22	Windows	Base
7	24	Ubuntu	Added with include

G.5.a Interview point

If you see a matrix, think: 'same job definition, multiple configurations'. Then calculate the combinations, apply include/exclude, and consider how failures should affect the rest of the run.

PART H | Reusable Workflows

Reuse complete workflow logic across repositories or projects.

H.1 Why reusable workflows?

A reusable workflow lets one workflow call another workflow so the same CI/CD logic does not have to be copied into many files.

```
Caller workflow
    |
    +--> build
    |
    +--> reusable deployment workflow
            |
            +--> download artifact
            +--> deploy
            +--> return result
```

H.1.a Reusable workflow vs composite action

Reusable workflow	Composite action
Reuses a complete workflow	Reuses a sequence of steps as one action
Called at job level	Used inside `steps` via `uses`
Can contain multiple jobs	Groups steps
Good for organization-wide CI/CD patterns	Good for repeated step sequences

H.2 Creating a reusable workflow

```yaml
# .github/workflows/reusable-deploy.yml

name: Reusable deploy

on:
  workflow_call:
    inputs:
      artifact-name:
        description: Name of the deployable artifact
        required: false
        default: dist
        type: string

    secrets:
      some-secret:
        required: false

    outputs:
      result:
        description: Result of the deployment
        value: ${{ jobs.deploy.outputs.outcome }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    outputs:
      outcome: ${{ steps.set-result.outputs.result }}

    steps:
      - name: Download artifact
        uses: actions/download-artifact@v5
        with:
          name: ${{ inputs.artifact-name }}

      - name: List files
        run: ls -la

      - name: Deploy
        run: echo "Deploying..."

      - name: Set result
        id: set-result
        run: echo "result=success" >> "$GITHUB_OUTPUT"
```

Dry run: another workflow calls this file -> GitHub passes `artifact-name` -> the reusable workflow downloads that artifact -> the inner job creates output `outcome` -> the workflow maps it to the reusable workflow output `result` -> the caller can read that result.

H.3 Calling a reusable workflow

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: dist-files
          path: dist

  deploy:
    needs: build
    uses: ./.github/workflows/reusable-deploy.yml
    with:
      artifact-name: dist-files
    secrets:
      some-secret: ${{ secrets.some-secret }}

  print-result:
    needs: deploy
    runs-on: ubuntu-latest
    steps:
      - run: echo "Result = ${{ needs.deploy.outputs.result }}"
```

Dry run: `build` uploads `dist-files` -> `deploy` calls the reusable workflow after `build` -> the called workflow returns `result` -> `print-result` reads that output.

H.4 Reusable workflow inputs

Inputs are named values passed from caller to callee. Modern GitHub expects you to declare an input's type such as string, boolean, or number.

H.5 Reusable workflow secrets

```
secrets:
  personal_access_token:
    required: true
```

The caller passes a secret with the `secrets` mapping. In the same organization/enterprise, `secrets: inherit` can be useful when you deliberately want the caller's secrets available to the called workflow.

H.6 Important rules

- A reusable workflow must include `workflow_call`.
- The reusable workflow lives under `.github/workflows/`.
- A reusable workflow is invoked at the job level, not from inside a normal step.
- The caller can pass `with`, `secrets`, `needs`, `strategy`, `if`, permissions, and other supported job-level keys.

Source correction: The source's `::set-output` example is updated to `$GITHUB_OUTPUT`. The source also contains small key-name typos such as `result` vs `results`; this PDF uses a consistent `result` name.

H.7 When should you use it?

Use reusable workflows when many repositories need the same multi-job pipeline. Example: 30 company services all need the same 'build -> security scan -> package -> deploy to staging' flow. Centralize the logic instead of copying 30 YAML files.

PART I | Jobs and Docker Containers

Use containers when you need a controlled execution environment or supporting services.

I.1 What is a container?

A container packages application code with its runtime and supporting files in a portable execution environment. A container is lighter than a full virtual machine because it shares the host kernel, while still isolating processes and filesystem changes.

I.1.a Why use containers in GitHub Actions?

- Reproduce a known runtime environment.
- Use an image that already contains the required tools.
- Reduce 'works on my machine' differences.
- Run integration tests against an isolated database or cache service.

I.1.b Runner vs container

Runner	Job container
GitHub chooses a hosted VM/image or you use a self-hosted runner.	Your job executes inside a specified container image.
Convenient predefined tools.	Greater control over the job environment.
Good for general CI tasks.	Good when exact runtime dependencies matter.

I.2 Dockerfile basics

A Dockerfile defines how an image is built. The source example uses a Node image, environment variables, a working directory, copied files, dependency installation, and a start command.

```
FROM node:20

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .

CMD ["npm", "start"]
```

Dry run: `FROM` selects the base image -> `WORKDIR` sets the application directory -> `COPY package*.json ./` copies dependency manifests -> `RUN npm ci` installs dependencies during image build -> `COPY . .` adds source code -> `CMD` defines the default process.

Security correction: Do not put real database passwords or API keys in Dockerfile `ENV` instructions. Image layers can preserve those values and anyone with the image may be able to inspect them. Supply secrets at runtime instead.

I.3 Running a job in a container

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    container:
      image: node:20

    steps:
      - uses: actions/checkout@v6
      - run: npm ci
      - run: npm test
```

The runner hosts the container, but the job's `run` steps execute inside the specified container.

> Platform rule: GitHub documents that Docker container jobs, Docker container actions, and service containers require a Linux runner. With GitHub-hosted runners, use Ubuntu.

I.4 Service containers

A service container is a helper container that provides a database, cache, queue, or similar service to the main job. The classic use case from the source is a testing database that should not be the production database.

```
Job container / runner
        |
        +---- application tests
        |
        +---- service container: MongoDB
        |
        +---- service container: Redis
```

I.4.a Example: MongoDB service

```yaml
jobs:
  test:
    runs-on: ubuntu-latest

    container:
      image: node:20

    services:
      mongodb:
        image: mongo:8

    steps:
      - uses: actions/checkout@v6
      - run: npm ci

      - name: Run tests
        env:
          MONGODB_HOST: mongodb
          MONGODB_PORT: 27017
        run: npm test
```

Dry run: GitHub starts the job container and MongoDB service container -> both join the same Docker network for the job -> inside the job container, hostname `mongodb` points to the MongoDB service -> tests connect without touching production.

I.5 Networking: container job vs runner job

Job runs...	How to reach service
Inside a job container	Use the service label as hostname, e.g. `mongodb:27017`.
Directly on runner	Map a host port and use `localhost:<port>` or `127.0.0.1:<port>`.

This distinction is frequently tested in interviews because `localhost` means the current network namespace. When your test process itself runs inside a container, `localhost` points to that job container, not automatically to the database service container.

I.6 Original source idea: test database

The source's scenario is: run tests -> problem: tests should not manipulate the production database -> solution: use a testing database service container. This is a strong real-world pattern for integration tests.

I.7 Container checklist

- Choose a stable base image/version.
- Keep credentials out of the image.
- Understand the job's shell; containers may use `sh` by default.
- Use service labels as hostnames for container-to-container communication.
- Use health checks or wait strategies when the service must be ready before tests start.

PART J | Building and Using Custom Actions

Package repeated or specialized automation into reusable actions.

J.1 Why custom actions?

- Simplify workflows by replacing several repeated steps with one reusable action.
- Solve a specialized task when no public action fits.
- Share common automation with a team or community.

J.1.a Real-world example

Suppose every Node.js project in your company needs the exact same 'restore npm cache -> install dependencies -> report cache hit' logic. Build one custom action and call it as a single workflow step.

J.2 Three types of custom actions

Type	Idea	Best fit
Composite	Combine multiple workflow steps.	Shared `run` + `uses` sequences.
JavaScript	Run JavaScript/Node action code.	Logic that benefits from GitHub Actions toolkit APIs.
Docker	Build a custom Docker execution environment.	Any language/tooling you can package in a container.

J.3 Composite action

A composite action groups multiple workflow steps behind one `uses` call. The `action.yml` file is the metadata definition.

```
# .github/actions/cached-deps/action.yml

name: Get and Cache Dependencies
description: Get npm dependencies and cache them

inputs:
  caching:
    description: Whether to use dependency caching
    required: false
    default: 'true'

outputs:
  used-cache:
    description: Whether the cache was used
    value: ${{ steps.cache.outputs.cache-hit }}

runs:
  using: composite
  steps:
    - name: Cache dependencies
      id: cache
      if: inputs.caching == 'true'
      uses: actions/cache@v4
      with:
        path: ~/.npm
        key: deps-node-${{ hashFiles('**/package-lock.json') }}

    - name: Install dependencies
      if: steps.cache.outputs.cache-hit != 'true' || inputs.caching != 'true'
      run: npm ci
      shell: bash
```

Dry run: input `caching` defaults to true -> cache step runs -> if cache hit is true, `npm ci` is skipped -> if cache miss, `npm ci` runs -> output `used-cache` exposes the cache-hit result.

Source correction: The supplied composite-action output points at `steps.install.outputs.cache`, but the install step does not create such an output. The corrected example maps the output from the cache step itself.

J.3.a Use the composite action

```
- name: Load and cache dependencies
  uses: ./.github/actions/cached-deps
```

The action folder path points to a directory containing `action.yml`. The source's instruction to create the folder and use a local path is correct.

J.4 JavaScript action

A JavaScript action packages Node.js logic and can use the Actions toolkit. The source mentions `@actions/core`, `@actions/github`, and `@actions/exec`. These libraries provide helpers for inputs, GitHub APIs, command execution, logs, outputs, and other workflow operations.

J.4.a Metadata

```
name: Deploy to AWS S3
description: Deploy a static website to Amazon S3

inputs:
  bucket:
    description: The S3 bucket name
    required: true

  bucket-region:
    description: The region of the S3 bucket
    required: false
    default: us-east-1

  dist-folder:
    description: Folder containing deployable files
    required: true

outputs:
  website-url:
    description: URL of the deployed website

runs:
  using: node24
  main: main.js
```

Current GitHub documentation uses modern Node action runtimes such as node24 and recommends packaging the action's executable code appropriately for production distribution. Older course material may use node16; treat that as historical syntax, not current guidance.

J.5 JavaScript implementation - S3 concept

```
const core = require('@actions/core');
const exec = require('@actions/exec');

async function run() {
  const bucket = core.getInput('bucket', { required: true });
  const bucketRegion = core.getInput('bucket-region', { required: true });
  const distFolder = core.getInput('dist-folder', { required: true });

  const s3Uri = `s3://${bucket}`;

  await exec.exec(
    `aws s3 sync "${distFolder}" "${s3Uri}" --region "${bucketRegion}"`
  );

  const websiteUrl =
    `http://${bucket}.s3-website-${bucketRegion}.amazonaws.com`;

  core.notice('Deployment completed.');
  core.setOutput('website-url', websiteUrl);
}

run().catch(error => core.setFailed(error.message));
```

Dry run: `getInput` reads values from the workflow -> `s3Uri` becomes something like `s3://my-static-site` -> `aws s3 sync` copies the static files -> the code constructs the website URL -> `core.setOutput` exposes `website-url` for the next workflow step.

> Prerequisite: The AWS CLI must exist in the action's execution environment, or the action must use an AWS SDK instead. A custom JavaScript action does not magically install the CLI.

J.6 Node packages used by the source

```
npm install @actions/core @actions/github @actions/exec
```

Use the packages you actually need. In a production action, keep dependency versions controlled and package the action as recommended by current GitHub guidance.

J.7 Docker action

A Docker action packages the action's environment into a container image. This lets you use another language, such as Python, and install system packages you need.

J.7.a Safer current structure

```
# action.yml
name: Deploy to AWS S3
description: Deploy a static website via AWS S3

inputs:
  bucket:
    description: The S3 bucket name
    required: true

  bucket-region:
    description: The region of the S3 bucket
    required: false
    default: us-east-1

  dist-folder:
    description: The deployable folder
    required: true

outputs:
  website-url:
    description: URL of the deployed website

runs:
  using: docker
  image: Dockerfile
  args:
    - ${{ inputs.bucket }}
    - ${{ inputs.bucket-region }}
    - ${{ inputs.dist-folder }}
```

This uses action metadata inputs and passes them as Docker arguments. GitHub documents this pattern for Docker actions.

J.7.b Dockerfile

```
FROM python:3.12-slim

COPY requirements.txt /requirements.txt
RUN pip install --no-cache-dir -r /requirements.txt

COPY deployment.py /deployment.py

ENTRYPOINT ["python", "/deployment.py"]
```

J.7.c Python deployment code

```
import sys
import boto3
from botocore.config import Config

def run():
    bucket = sys.argv[1]
    bucket_region = sys.argv[2]
    dist_folder = sys.argv[3]

    config = Config(region_name=bucket_region)
    s3_client = boto3.client("s3", config=config)

    # Simplified example: upload files recursively.
    import os
    for root, _, files in os.walk(dist_folder):
        for file in files:
            full_path = os.path.join(root, file)
            key = os.path.relpath(full_path, dist_folder)
            s3_client.upload_file(full_path, bucket, key)

    website_url = (
        f"http://{bucket}.s3-website-{bucket_region}.amazonaws.com"
    )
    with open(os.environ["GITHUB_OUTPUT"], "a") as fh:
        print(f"website-url={website_url}", file=fh)

if __name__ == "__main__":
    run()
```

Important teaching note: the source uses environment names such as `INPUT_BUCKET` and a `::set-output` command. The modern, clearer pattern is to pass inputs as Docker `args` and publish outputs through the GitHub output file mechanism. In a real Docker action, the output file path is supplied by GitHub via the `GITHUB_OUTPUT` environment variable, so the final code should write to that environment path.

J.7.d Requirements

```
boto3
botocore
```

The source lists exact pinned versions for boto3, botocore, jmespath, python-dateutil, s3transfer, six and urllib3. Those are historical course-example pins; for current work, generate or verify a compatible lock/pin set rather than copying old versions blindly.

J.8 JavaScript vs Docker vs Composite - decision guide

Question	Choose
Am I just combining existing actions and shell commands?	Composite
Do I need custom logic with GitHub toolkit APIs?	JavaScript
Do I need a special execution environment or another language?	Docker
Do I need to reuse several jobs as a complete workflow?	Reusable workflow

J.9 Share a custom action

- Keep the action in a repository or a folder under your repository.
- Document inputs, outputs and usage in a README.
- Tag releases so users can reference a stable version such as `v1`.

- For stronger supply-chain security, pin third-party actions to full commit SHAs when practical.

PART K | Security and Permissions

Automation is powerful because it can read code, use secrets, and make changes. That is exactly why it must be secured.

K.1 Main security concerns from the source

Risk	What can go wrong?
Script injection	Untrusted issue/PR text is interpreted as shell code.
Malicious third-party action	An action can read data available to the job and potentially abuse tokens.
Overly broad permissions	The workflow gets more access than it actually needs.

K.2 Script injection

Script injection happens when a value supplied by an attacker flows directly into a shell command or another interpreter in a way that changes the command being executed.

K.2.a Unsafe pattern

```
- name: Check issue title
  run: |
    issue_title="${{ github.event.issue.title }}"

    if [[ "$issue_title" == *"bug"* ]]; then
      echo "Issue is about a bug!"
    else
      echo "Issue is not about a bug"
    fi
```

Why unsafe? The expression is substituted before the shell runs. A malicious issue title can introduce shell metacharacters or commands.

K.2.b Conceptual attack

The source demonstrates an attacker entering an issue title that closes the quoted string and appends another command. The exact payload does not matter for the lesson; the key idea is that attacker-controlled text became executable shell syntax.

K.2.c Safer pattern

```
- name: Check issue title safely
  env:
    TITLE: ${{ github.event.issue.title }}
  run: |
    if [[ "$TITLE" == *"bug"* ]]; then
      echo "Issue is about a bug!"
    else
      echo "Issue is not about a bug"
    fi
```

Here the untrusted value is passed into an environment variable first. The shell then treats `$TITLE` as data rather than directly splicing the GitHub expression into the command text.

Rule: Treat event fields such as issue titles, PR titles/bodies, branch names and similar external input as untrusted.

K.3 Third-party actions

An action can run code with the privileges available to its job. A compromised or malicious action can potentially inspect workspace files, environment variables, tokens, or secrets that the job can access.

K.3.a Practical trust ladder

```
Higher trust
|
+-- Your own audited action
|
+-- Verified/trusted publisher
|
+-- Well-known maintained community action
|
+-- Unknown third-party action
|
Lower trust
```

This is a mental model, not a formal GitHub rating. The stronger practice is to review what an action does, pin the reference appropriately, and give it only the permissions it needs.

K.4 Pin actions

GitHub's security guidance recommends pinning actions to full-length commit SHAs when you need an immutable action reference. Tags can move; a full SHA identifies a specific commit.

```
- uses: actions/checkout@<full-40-character-commit-sha>
```

For learning, version tags such as `@v6` are easier to read. For higher-security production pipelines, consider SHA pinning and automate updates with Dependabot or an equivalent control.

K.5 Permissions and least privilege

`GITHUB_TOKEN` is a repository-scoped token automatically created for each workflow job. You can restrict its access with the `permissions` key. The core principle is: grant only what the job needs.

```
name: Label issues

on:
  issues:
    types: [opened]

jobs:
  assign-label:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      issues: write

    steps:
      - name: Assign label
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          echo "Call GitHub API to add the bug label"
```

Dry run: the workflow can read repository contents and write issues, but it is not granted broad write permissions for unrelated resources.

K.5.a Permission values

For supported permission scopes, GitHub lets you specify `read`, `write`, or `none`. If you set specific permissions, unspecified permissions are restricted according to GitHub's documented rules.

K.5.b Important source correction

Correction: The source suggests generating a personal token and enabling GitHub Actions permissions for ordinary workflow updates. Modern GitHub guidance says to use the built-in `GITHUB_TOKEN` whenever possible, with explicit permissions. Use a PAT or GitHub App only when you genuinely need capabilities outside that token's scope.

K.6 Workflow settings related to security

- Restrict which actions and reusable workflows are allowed.
- Control whether workflows can create/approve pull requests.
- Require approval for first-time contributors where appropriate.
- Use environment protection rules for production.
- Prefer least privilege for `GITHUB_TOKEN`.

K.7 OpenID Connect (OIDC)

For cloud deployments, OIDC lets a workflow authenticate to a cloud provider using short-lived identity tokens instead of storing long-lived cloud credentials as GitHub secrets.

```
permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsDeploy
          aws-region: us-east-1

      - name: Deploy
        run: aws s3 sync dist s3://my-bucket
```

Dry run: `id-token: write` allows the workflow to request an OIDC token -> AWS trusts the configured GitHub identity subject/claims -> the action exchanges that identity for temporary AWS credentials -> the deploy command uses those temporary credentials.

Security rule: OIDC is safer than long-lived cloud keys only when the cloud trust policy is tightly scoped. Configure conditions so an untrusted repository or branch cannot assume the deployment role.

K.8 Security example: full flow

```
Issue opened
    |
    +--> Event title is untrusted input
    |
    +--> workflow receives title via env
    |
    +--> job has issues: write only
    |
    +--> trusted/pinned actions only
    |
    +--> secrets not printed
    |
    +--> production deployment uses protected environment
    |
    +--> cloud access uses OIDC / short-lived credentials
```

K.9 Security checklist

- Never hard-code passwords/API keys in workflow YAML or Dockerfiles.
- Treat PR/issue/branch input as untrusted.
- Use `env` to pass untrusted values into shell code instead of direct string interpolation into commands.
- Minimize `GITHUB_TOKEN` permissions.
- Review and pin third-party actions when security requirements are high.
- Use protected environments for sensitive deployments.
- Prefer OIDC for cloud authentication when supported.
- Do not print secrets, tokens, or full sensitive contexts to logs.
- Rotate credentials if a secret is exposed.

PART L | Essential Additions for Modern GitHub Actions Practice

These topics were added because they make the source material safer, more current, and easier to apply in real projects.

L.1 Modern action-version awareness

Course materials age faster than the core concepts. The supplied material repeatedly references v3 of checkout/setup-node/cache/upload/download. Current GitHub documentation shows newer major releases, so the transferable lesson is: learn the syntax and purpose, then verify the action's current supported major version before production use.

Source-style example	Modernized learning example
actions/checkout@v3	actions/checkout@v6
actions/setup-node@v3	actions/setup-node@v7
actions/cache@v3	actions/cache@v4
actions/upload-artifact@v3	actions/upload-artifact@v4
actions/download-artifact@v3	actions/download-artifact@v5
::set-output	$GITHUB_OUTPUT

L.2 Concurrency

Concurrency is a useful production control when multiple runs of the same workflow can overlap. For example, if five pushes happen quickly, you may want to cancel older in-progress runs and keep the newest one.

```
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
```

Dry run: push 1 starts CI -> push 2 for the same branch arrives -> GitHub can cancel the older run and continue with the newer run. This prevents stale work from consuming runner time.

Added for completeness: Concurrency is especially useful for deployment workflows where an older deployment should not race a newer deployment.

L.3 Artifacts vs outputs vs cache

Feature	Think of it as	Lifetime / purpose
Output	A value	Pass a small value to later steps/jobs.
Artifact	A file package	Keep/share build/test files.
Cache	Reusable dependency store	Speed up future runs; not a deployment handoff mechanism.

L.3.a Interview memory

If the data is `version=1.2.3`, think output. If the data is `dist/` or a JAR, think artifact. If the data is npm cache or Gradle cache, think cache.

L.4 YAML basics you must know

- Indentation matters.
- Use spaces, not tabs.
- Keys such as `jobs`, `steps`, `with`, `env`, `permissions`, and `strategy` have specific locations.
- A one-space indentation error can change the meaning of the file.

L.4.a Minimal syntax map

```
name: Workflow name
on:
  push:

jobs:
  job-id:
    runs-on: ubuntu-latest
    steps:
      - name: Step name
        uses: actions/checkout@v6

      - name: Shell command
        run: npm test
```

L.5 Troubleshooting workflow failures

Problem	First thing to inspect
Workflow does not start	`on` trigger, branch filter, path filter, workflow file location.
Job skipped	`needs` dependency result or job-level `if`.
Step skipped	Previous step failure or step-level `if`.
Command not found	Runner/container image and installed tools.
Second job cannot find files	Remember jobs use separate runners; use artifacts or regenerate files.
Cache never hits	Cache path, key, lock-file hash, OS, branch scope.
Secret is empty	Secret name, environment, fork/approval restrictions, permission model.
Docker service not reachable	Container networking and service hostname/ports.
Deployment unexpectedly runs	Trigger filters, environments, permissions, reusable workflow caller.

L.6 End-to-end CI/CD blueprint

```yaml
name: Production pipeline

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:

concurrency:
  group: prod-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v7
        with:
          node-version: '20'
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test

  build:
    needs: test
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.meta.outputs.version }}
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v7
        with:
          node-version: '20'
          cache: npm
      - run: npm ci
      - run: npm run build

      - id: meta
        run: echo "version=${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"

      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist

  deploy:
    needs: build
    environment: production
    permissions:
      contents: read
      id-token: write
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v5
        with:
          name: dist

      - name: Configure cloud identity
        run: echo "Configure OIDC-based cloud credentials here"

      - name: Deploy
        run: echo "Deploy version ${{ needs.build.outputs.version }}"
```

Dry run: PR -> test only. Push to main -> test -> build -> artifact upload -> protected production deployment. Manual run -> behavior depends on the event-specific conditions you define. Concurrency prevents stale overlapping runs on the same reference.

L.7 Final revision sheet

Question	Answer in one line
What is Git?	Version control system.
What is GitHub?	Cloud platform around Git repositories and collaboration.
What is an Action?	Reusable automation component.
What is a Workflow?	YAML automation process.
What is a Job?	Unit of work running on a runner.
What is a Step?	One action or shell command inside a job.
What is `runs-on`?	Selects the runner environment.
What is `needs`?	Defines a job dependency.
What is `if`?	Controls conditional execution.
What is a context?	Structured workflow metadata/data object.
What is an artifact?	Saved files/folders from a run.
What is an output?	A value passed to later automation.
What is a cache?	Reusable data to speed future runs.
What is `env`?	Environment variable configuration.
What is `secrets`?	Sensitive values stored securely.
What is an environment?	Deployment-scoped config and protection.
What is a matrix?	Many configurations from one job definition.
What is `workflow_call`?	Trigger used to make a workflow reusable.
What is a service container?	Supporting container such as a DB or Redis.
What is OIDC?	Short-lived cloud identity federation.

L.8 Professor-style exam questions

- Draw the relationship between workflow, jobs, steps, runners and events.
- Explain why two jobs cannot automatically see each other's files.
- Write a workflow that runs only on pushes to main.
- Write a pull_request workflow that runs only when a PR is opened or reopened.
- Explain the difference between artifact, output and cache with examples.
- Dry-run a `needs` graph with parallel and sequential jobs.
- Explain `failure()`, `success()`, `always()` and `cancelled()`.
- Dry-run a 3 x 2 matrix with one include and one exclude.
- Explain how a reusable workflow receives inputs, secrets and outputs.
- Explain the difference between a job container and a service container.
- Identify the script-injection bug in a workflow that interpolates an issue title directly into `run`.
- Explain least privilege for `GITHUB_TOKEN`.

- Why is OIDC preferable to long-lived AWS credentials in a CI/CD pipeline?

L.9 One-page mental model

```
EVENT
  |
  v
WORKFLOW
  |
  +--> JOB 1 ----> STEP 1 -> STEP 2 -> STEP 3
  |
  +--> JOB 2 ----> STEP 1 -> STEP 2
          ^
          |
        needs
          |
     JOB 1 result

Inside jobs you can use:
  - actions (`uses`)
  - shell commands (`run`)
  - environment variables (`env`)
  - secrets (`secrets`)
  - contexts / expressions
  - conditions (`if`)
  - caches
  - artifacts
  - containers / services
  - matrix configurations

For production:
  + least privilege
  + trusted/pinned actions
  + protected environments
  + safe handling of untrusted input
  + OIDC for cloud access
  + concurrency where stale runs should be cancelled
```

L.10 Official references used for the modernization pass

GitHub Docs topics checked while updating this study note: workflow syntax; workflow events; contexts; workflow commands and $GITHUB_OUTPUT; dependency caching; workflow artifacts; variables/secrets/environments; reusable workflows; job/service containers; custom action metadata; secure use reference; GITHUB_TOKEN; and OIDC in AWS.

Reference area	Why it was used
GitHub workflow syntax	Permissions, events, env, job outputs and modern YAML behavior.
GitHub events reference	Activity types and trigger behavior.
GitHub artifacts docs	Current upload/download artifact examples.
GitHub cache docs	Cache keys, restore keys, cache-hit behavior.
GitHub environments/variables	Environment scoping and configuration values.
Reusable workflows	workflow_call, inputs, secrets, outputs and caller rules.
Container docs	Job/service containers and networking.
Custom action docs	Composite, JavaScript and Docker action metadata.
Security/OIDC docs	Script injection, token permissions, action supply chain, cloud identity.